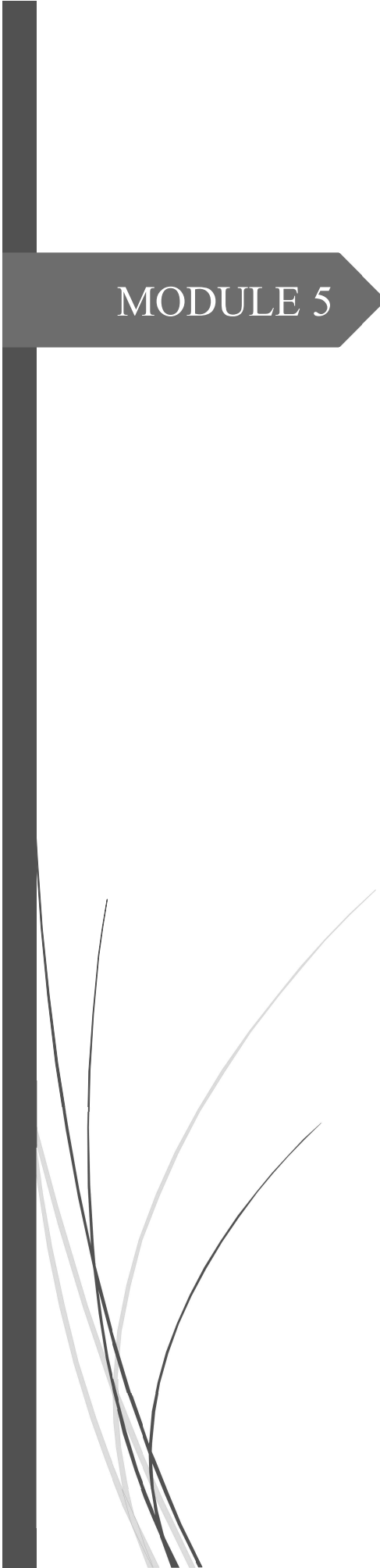




MODULE 5

APPLICATION LAYER

TOPICS COVERED:

- ✓ 5.1 Application Layer: Application Layer Paradigms, Application ProgrammingInterface (Socket and socket address)
- ✓ 5.2 Standard Client Server Protocols: World Wide Web
- ✓ 5.3 HTTP
- ✓ 5.4 FTP
- ✓ 5.5 Electronic Mail
- ✓ 5.6 TELNET
- ✓ 5.7 Secure Shell (SSH)- components, Domain Name System (DNS)

Ms. BHUVANA S
ASSISTANT PROFESSOR, DEPT. OF CSE

Module 5: Application Layer

5.1 Application-Layer Paradigms

To utilize the Internet effectively, two application programs on different computers (often separated by vast geographical distances) must interact with one another. These programs utilize the underlying Internet infrastructure to exchange messages. However, a fundamental question arises regarding the architectural relationship between these two programs:

- *Should both programs be equal?*
- *Should one provide service while the other requests it?*

To answer this, the Internet architecture has evolved two primary paradigms:

1. The **Client-Server** Paradigm.
2. The **Peer-to-Peer (P2P)** Paradigm.

1. The Traditional Paradigm: Client-Server

For many years, the client-server model was the dominant and almost exclusive method of communication on the Internet. It defines a strict hierarchy and separation of concerns between the communicating entities.

Core Concepts

- **Service Provider (Server):**
 - This is an application program known as the **server process**.
 - **Status:** It runs **continuously** (24/7/365). It must be "always-on."
 - **Function:** It waits passively for other applications to contact it.
 - **Capacity:** A server is usually a powerful computer designed to handle requests from many clients simultaneously.
- **Service Requester (Client):**
 - This is an application program known as the **client process**.
 - **Status:** It is started only when the user needs a specific service.

- **Function:** It waits passively for other applications to contact it.
- **Capacity:** A server is usually a powerful computer designed to handle requests from many clients simultaneously.
- **Service Requester (Client):**
 - This is an application program known as the **client process**.
 - **Status:** It is started only when the user needs a specific service.
 - **Function:** It actively initiates a connection across the Internet to the server to request a service.
- **Relationship:** The relationship is **many-to-one**. Many clients request services from a few servers.

Operational Flow

1. The **Server process** starts and initializes. It waits for incoming requests.
2. The **Client process** starts (triggered by a user).
3. The Client initiates a connection through the Internet to the Server.
4. The Client sends a request; the Server processes it and sends a response.

Real-World Analogy: The Telephone Directory

The text provides a helpful analogy to understand this relationship:

- **The Server:** A telephone directory center. It must be open and available all the time to answer queries.
- **The Client:** A subscriber who needs a phone number. They only call the center when they need information for a short duration.
- **Key Distinction:** The subscriber never acts as the directory center, and the center never acts as a subscriber. The roles are fixed.

Limitations of Client-Server

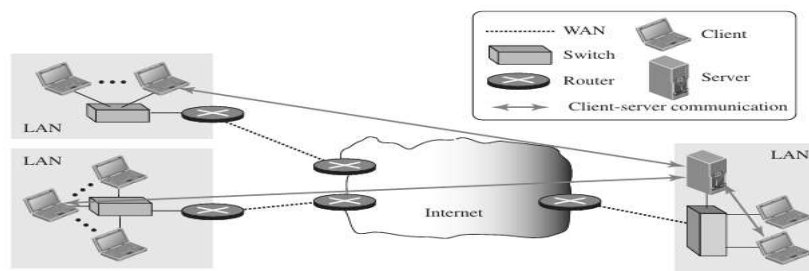
While reliable, this paradigm faces specific challenges:

1. **Load Concentration:** All communication load is shouldered by the server. If thousands of clients connect at once, the server can become a bottleneck.
2. **Hardware Requirements:** Because of the high load, the server must be a powerful,

3. expensive machine.
4. **Cost:** A service provider must be willing to bear the cost of maintaining this powerful infrastructure, usually requiring a business model where the service generates income.

Examples: World Wide Web (HTTP), File Transfer Protocol (FTP), Secure Shell (SSH), Email.

Figure 25.2 Example of a client-server paradigm



Scenario 1: Analyzing Figure 25.2 (Client-Server Paradigm)

Context:

The diagram shows multiple Local Area Networks (LANs) and Wide Area Networks (WANs) connected via the Internet. There are specific machines labelled "Client" and machines labelled "Server."

Analysis Steps:

1. **Identification:** We see multiple laptops (Clients) on the left side. We see a tower computer (Server) on the right side.
2. **Initiation:** The blue arrow originates from the Client. This signifies that the **Client initiates the contact**. The server does not call the client.
3. **Routing:** The request travels through the local switch, through the router, across the "Internet" cloud, to the destination router, and finally to the Server.
4. **Load Imbalance:** Notice that if multiple clients (the three laptops shown) all send requests simultaneously, the single link leading to the Server becomes a funnel. The Server must process all three requests.
5. **Role Fixation:** The laptops are only sending requests. The server is only receiving them (and sending replies). They do not swap roles.

2. The New Paradigm: Peer-to-Peer (P2P)

To address the scalability and cost issues of the client-server model, and to facilitate new types of applications, the Peer-to-Peer paradigm emerged.

Core Concepts

- **Equality:** There is no central server process that must run all the time.
- **Shared Responsibility:** The responsibility for providing the service is distributed among the **peers** (the computers connected to the network).
- **Dual Roles:** A defining characteristic of P2P is that a computer can act as **both a client and a server simultaneously**. It can request a service from one peer while providing a service to another peer at the same time.

Operational Flow

1. Peers connect to the Internet.
2. Peers locate other peers (discovery).
3. Peers communicate directly without passing data through a centralized server.

Advantages

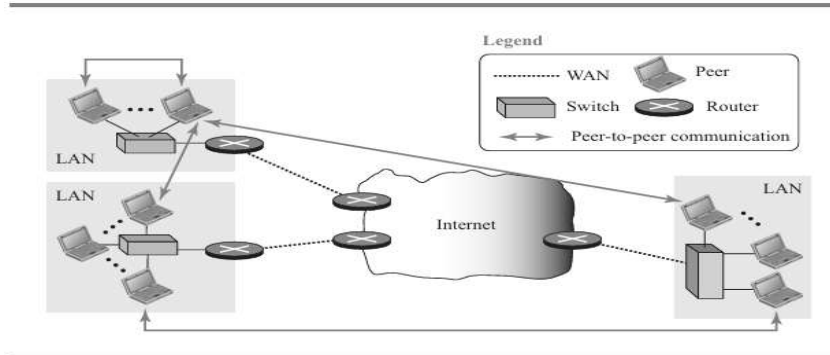
- **Scalability:** The system scales easily. As more peers join, the capacity of the system actually increases (because the new peer brings new resources), whereas in client-server, new users decrease the system's performance (by adding load).
- **Cost-Effective:** It eliminates the need for expensive, dedicated "always-on" servers.

Challenges

- **Security:** This is the most significant challenge. It is much harder to secure a distributed network where every node is a potential server than it is to secure a centralized server farm.
- **Incentive:** Users may not want to share their resources (bandwidth/storage).
- **Applicability:** Not all applications fit this model (e.g., search engines like Google require centralized indexing).

Examples: BitTorrent (file sharing), Skype (VoIP), IPTV, Internet Telephony.

Figure 25.3 Example of a peer-to-peer paradigm



Scenario 2: Analyzing Figure 25.3 (Peer-to-Peer Paradigm)

Context:

The diagram shows a similar network topology, but the communication arrows are different.

Analysis Steps:

1. **Identification:** All machines are labelled "Peer." There is no dedicated "Server."
2. **Connectivity:**
 - Peer A (top left) connects to Peer B (bottom left).
 - Peer A (top left) also connects to Peer C (right side).
3. **Simultaneity:** Notice the arrows are double-headed or point in multiple directions. Peer A might be downloading a file from Peer C (acting as a client) while simultaneously uploading a different file to Peer B (acting as a server).
4. **Load Distribution:** The traffic is not directed toward a single point. It is spread across the "Internet" cloud.
5. **Independence:** If Peer C goes offline, Peer A and Peer B can continue to communicate. In the Client-Server model, if the Server goes offline, the entire service stops.

3. The Mixed Paradigm

In modern networking, applications often do not stick strictly to one model. They use a **hybrid approach**.

- **Concept:** Combine the advantages of both paradigms.
- **Implementation:**

1. Use a **Client-Server** model for **discovery** (finding out where other users are or where a file is located). This is a "light-load" operation suitable for a server.
 2. Use a **Peer-to-Peer** model for the **actual service** (transferring the heavy files or streaming audio/video).
- **Example:** A user connects to a central server to find the IP address of a friend (Client-Server), but the voice call itself happens directly between the two users (P2P).

Summary Comparison Table

Feature	Client-Server Paradigm	Peer-to-Peer Paradigm
Service Provider	Dedicated Server Process	Any Peer (Computer)
Initiator	Client always initiates	Any Peer can initiate
Availability	Server must be always-on	Peers connect intermittently
Scalability	Expensive; Server is a bottleneck	High; New peers add capacity
Security	Easier to manage (Centralized)	Difficult (Distributed)
Primary Cost	Provider pays for server infrastructure	Cost is shared among users' hardware
Examples	Web (HTTP), Email	BitTorrent, Skype

Client-Server Programming

In the client-server paradigm, communication at the application layer occurs between two distinct running application programs, referred to as **processes**: a *client* and a *server*.

- **The Client Process:**
 - It is a running program that initializes the communication.
 - It sends a request to the server.
 - **Lifecycle:** Finite. It starts when the user needs a service, sends a finite number of requests, receives responses, and then stops.
- **The Server Process:**
 - It is a running program that waits passively for requests.
 - It receives a request from a client, processes it, prepares a result, and sends

- back.
- **Lifecycle:** Infinite. Ideally, a server program starts and runs forever, ready to handle requests from any client at any time.
- **Important Constraint:** The server program must be started *before* any client attempts to connect to it.

Although we often speak of "client computers" and "server computers," the communication strictly happens between two **processes** running on those computers. It is even possible for one computer to run a client process and a server process simultaneously.

Application Programming Interface (API)

How does an application process actually send data? Computer programs are written in languages (like C, Java, Python) that have built-in instructions for math, string manipulation, and I/O. However, standard programming languages do not inherently know how to speak to the Internet.

To enable communication across a network, we need a set of instructions that tells the underlying operating system (which manages the TCP/IP protocol suite) to:

1. Open a connection.
2. Send data to the other end.
3. Receive data.
4. Close the connection.

This set of instructions is called an **Application Programming Interface (API)**. An API serves as the bridge between the application layer (the process) and the operating system (which encapsulates the Transport, Network, Datalink, and Physical layers).

Common APIs:

- **Socket Interface:** Started in the early 1980s at UC Berkeley for UNIX. It is the most common API and the focus of this chapter.
- Transport Layer Interface (TLI).
- STREAM.

Figure 25.4 Position of the socket interface

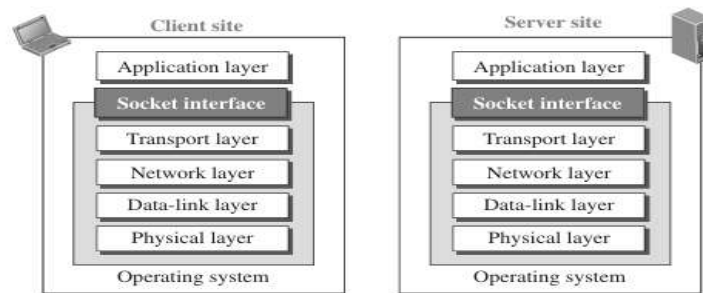
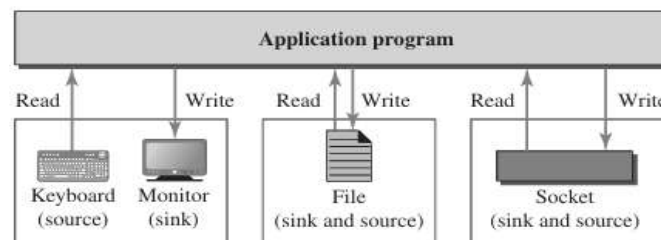


Figure 25.5 Sockets used the same way as other sources and sinks



The Socket Interface

A **socket** is an abstraction. It is a software object created by the application program that acts as an endpoint for communication.

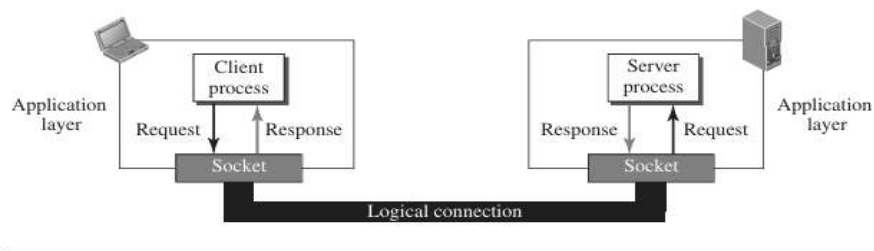
- **Analogy:** Just as a program can read from or write to a file (a source/sink of data), it can read from or write to a socket. The program sends data to the socket, and the operating system takes care of transmitting that data across the network. Conversely, the operating system receives data from the network and places it in the socket for the program to read.
- **Implementation:** The idea is to use the same standard I/O instructions found in languages like C or Java to interact with sockets, treating them just like files or terminals.

Communication Flow:

As depicted in Figure 25.6:

1. The **Client Process** creates a socket.
2. The **Server Process** creates a socket.
3. Communication is established between these two sockets. To the application, it looks like a direct logical connection.

Figure 25.6 Use of sockets in process-to-process communication



Socket Addresses

Communication in the client-server model is **process-to-process**. Therefore, to establish a connection, we need to identify the two endpoints uniquely. This requires a pair of addresses:

1. **Local Address (Sender)**
2. **Remote Address (Receiver)**

Since a socket defines an endpoint, it needs a specific address called a **Socket Address**. A socket address is a combination of two identifiers:

1. **IP Address (32-bit):** Identifies the unique computer (host) on the Internet.
2. **Port Number (16-bit):** Identifies the unique process (application) running on that computer.

Formula: Socket Address = IP Address + Port Number

Figure 25.7 A socket address



This combination allows the operating system to route an incoming packet not just to the correct computer, but to the exact application window or service waiting for it.

Finding Socket Addresses

The client and server have different ways of determining these necessary addresses.

1. Server Site

- **Local Socket Address (Server's own):**
 - **IP:** Provided by the operating system (the server machine's IP).
 - **Port:** A server process must use a standard, fixed port number so clients know where to find it. These are called **Well-Known Ports** (e.g., port 80 for HTTP, port 25 for SMTP).
- **Remote Socket Address (Client's):**
 - The server does not know this in advance. It obtains this information from the initial request packet sent by the client. The server extracts the client's IP and port from the request header to send the response back.

2. Client Site

- **Local Socket Address (Client's own):**
 - **IP:** Provided by the operating system.
 - **Port:** The client does not need a fixed port. The operating system assigns a temporary, random 16-bit integer called an **Ephemeral Port** (e.g., 52,000) for the duration of the connection.
- **Remote Socket Address (Server's):**
 - **Port:** The client *must* know the server's port number in advance (e.g., it knows to use port 80 for a web page).
 - **IP:** This is the challenge. Humans remember names (e.g., www.google.com), not IP addresses.
 - **Solution:** The **Domain Name System (DNS)**.
 - When a user types a URL or email address, the client program uses DNS to translate that human-readable name into the server's 32-bit IP address.
 - This is analogous to looking up a person's name in a telephone directory to find their phone number.

Summary Table: Socket Address Discovery

Site	Local Address (Who am I?)	Remote Address (Who am I talking to?)
Server	IP: OS provided. Port: Well-Known (fixed).	IP & Port: Extracted from the incoming request packet.
Client	IP: OS provided. Port: Ephemeral (assigned by OS).	IP: Found via DNS lookup. Port: Well-Known (known in advance).

5.2 Standard Client-Server Protocols**World Wide Web (WWW)**

The World Wide Web (WWW), proposed by Tim Berners-Lee at CERN in 1989, is a distributed repository of information spread across the globe. Its power and growth stem from two key concepts:

1. **Distributed:** Documents (web pages) are not stored in a single location but are hosted on thousands of separate servers worldwide. This prevents overloading a central server.
2. **Linked:** Documents are interconnected using **hypertext** and **hypermedia**.
 - **Hypertext:** A system where a document contains links to other documents. Clicking a link automatically retrieves the referenced document, regardless of where it is stored.
 - **Hypermedia:** An extension of hypertext that includes not just text but also images, audio, and video.

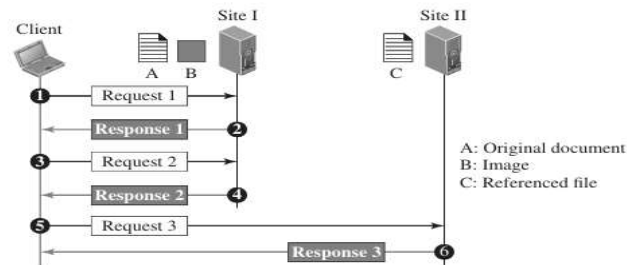
Architecture: The Web uses a distributed **client-server** architecture.

- **Client (Browser):** A program (like Chrome, Firefox) that retrieves and displays web pages.
- **Server:** A program that stores web pages and serves them to clients upon request.
- **Site:** A collection of related web pages stored on a specific server.
- **Web Page:** A file containing content (text, media) and links. It can be **simple** (text only) or **composite** (containing references to other files like images).

Example Scenario (Example 26.1):

Consider retrieving a web page (File A) that contains a reference to an image (File B) and a link to another text file (File C).

Figure 26.1 Example 26.1

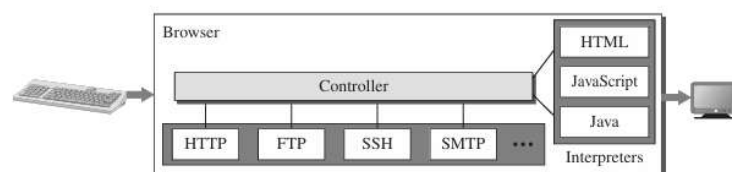


- Files A and B are on Server I.
- File C is on Server II.
- Transaction 1:** The client requests File A from Server I.
- Transaction 2:** The browser parses File A, sees the reference to Image B, and automatically sends a new request to Server I to retrieve Image B.
- Transaction 3:** If the user clicks the link to File C, the client sends a request to Server II to retrieve it.
- Note:** Each file is retrieved independently.

Components**1. Web Client (Browser):**

- Consists of a **Controller**, **Client Protocols** (HTTP, FTP), and **Interpreters** (HTML, Java, JavaScript).
- The controller takes input (clicks/URLs), uses a client protocol to fetch the document, and uses an interpreter to display it.

Figure 26.2 Browser



2. Web Server:

- Stores web pages.
- Uses efficient storage mechanisms (cache memory) and threading to handle multiple simultaneous requests.
- Examples: Apache, Microsoft IIS.

3. Uniform Resource Locator (URL):

- A unique identifier for every web page. It requires four pieces of information:
 1. **Protocol:** The vehicle for access (e.g., http, ftp).
 2. **Host:** The computer hosting the file (IP address or domain name like www.mhhe.com).
 3. **Port:** The entry point on the server (default for HTTP is 80).
 4. **Path:** The location and name of the file on the server's file system.
- **Standard Format:** protocol://host:port/path

Types of Web Documents

1. Static Documents:

- Fixed content created and stored on the server.
- The content is the same for every client.
- Created using languages like **HTML**, **XML**, **XHTML**.

2. Dynamic Documents:

- Created by the server *at the time of the request*.
- Used when content changes based on time, date, or request parameters (e.g., getting a timestamp).
- Technologies: **CGI** (Common Gateway Interface), **JSP** (Java Server Pages), **ASP** (Active Server Pages), **ColdFusion**, **PHP**.

3. Active Documents:

- A program or script that is run **at the client site** (in the browser).
- Used for animations or interactive features where the server cannot handle the processing load.
- Technologies: **Java Applets** (compiled bytecode), **JavaScript** (interpreted script).

5.3 HyperText Transfer Protocol (HTTP)

HTTP is the standard protocol used to fetch web pages. It is a **text-based, request-response** protocol.

- It uses the services of **TCP** on port **80**.
- It is **reliable** (because TCP is reliable).

Connections: Nonpersistent vs. Persistent

1. **Nonpersistent Connection (HTTP/1.0):**

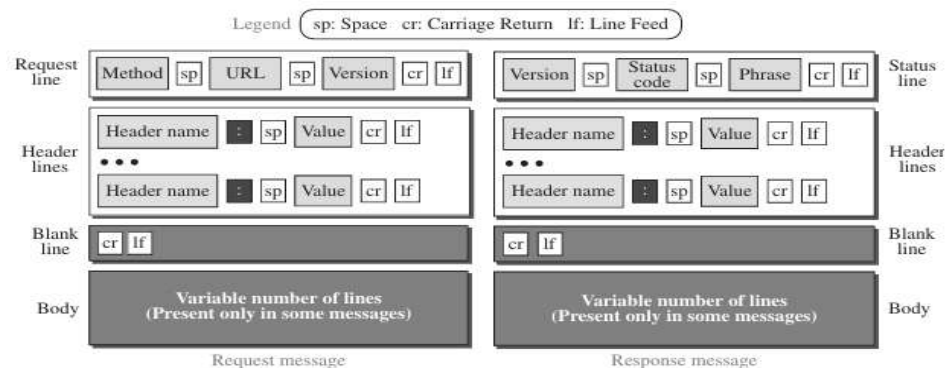
- **One TCP connection per object.**
- If a web page has one HTML file and 10 images, the client must open and close 11 separate TCP connections.
- **Overhead:** High. Establishing and terminating a TCP connection (3-way handshake) takes time and CPU resources for every single file.

2. **Persistent Connection (HTTP/1.1 Default):**

- **One TCP connection for multiple objects.**
- The server leaves the connection open after sending a response. The client can send subsequent requests (e.g., for the images embedded in the page) over the same open wire.
- **Efficiency:** Much higher. Reduces latency (fewer handshakes) and server load (fewer buffers/variables to manage).

Message Formats

Figure 26.5 *Formats of the request and response messages*



HTTP messages are either **Requests** (Client to Server) or **Responses** (Server to Client).

1. Request Message:

- **Request Line:** Method + URL + Version
 - **Method:** The action to be performed.
 - GET: Request a document (most common).
 - HEAD: Request only header info (no body).
 - POST: Send data to the server (e.g., form submission).
 - PUT: Upload a document to the server.
- **Headers:** Additional info (e.g., User-agent, Accept-language).
- **Body:** Input data (optional, used with POST/PUT).

Table 26.1 *Methods*

<i>Method</i>	<i>Action</i>
GET	Requests a document from the server
HEAD	Requests information about a document but not the document itself
PUT	Sends a document from the client to the server
POST	Sends some information from the client to the server
TRACE	Echoes the incoming request
DELETE	Removes the web page
CONNECT	Reserved
OPTIONS	Inquires about available options

2. Response Message:

- **Status Line:** Version + Status Code + Phrase
 - **Status Codes:**
 - **200 OK:** Successful request.
 - **301 Moved Permanently:** URL redirection.
 - **400 Bad Request:** Client error.
 - **404 Not Found:** Document doesn't exist.
 - **Headers:** Server info (e.g., Server, Content-Type, Content-Length).
- **Body:** The actual document (HTML, image, etc.).

Conditional Request

A client can ask for a document only if it has changed since a specific time.

- **Header:** If-Modified-Since: <date>

- **Server Response:**
 - If modified: Sends new document (200 OK).
 - If not modified: Sends 304 Not Modified (no body). This saves bandwidth.

Cookies

The **World Wide Web** was **originally stateless**, meaning that once a server responded to a client's request, it did not retain any memory of that interaction. While this design simplified server architecture, it made it impossible to support features that require continuity, such as **user authentication, shopping carts, and personalized content**. To overcome this limitation, the **HTTP Cookie mechanism** was introduced.

How Cookies Work

1. Creation

When a server receives a request from a client for the first time, it identifies the client as new. The server then generates a **unique identifier** (called a cookie value), which represents that client's session or identity.

2. Storage

The server sends this identifier to the client using the HTTP response header, typically in the form: *Set-Cookie: sessionId=12343*. The web browser stores this cookie locally, either in memory (session cookies) or on disk (persistent cookies), depending on its configuration.

3. Usage

For every subsequent request made by the browser to the same server, the stored cookie is automatically attached to the request header, for example: *Cookie: sessionId=12343*. This happens transparently without user intervention.

4. Recognition

Upon receiving the request, the server reads the cookie value and uses it as a key to retrieve the associated user data (such as login status, shopping cart items, or preferences) from its database or session store. In this way, the server can **maintain state across multiple requests**.

Uses of Cookies:

- **Electronic Commerce**

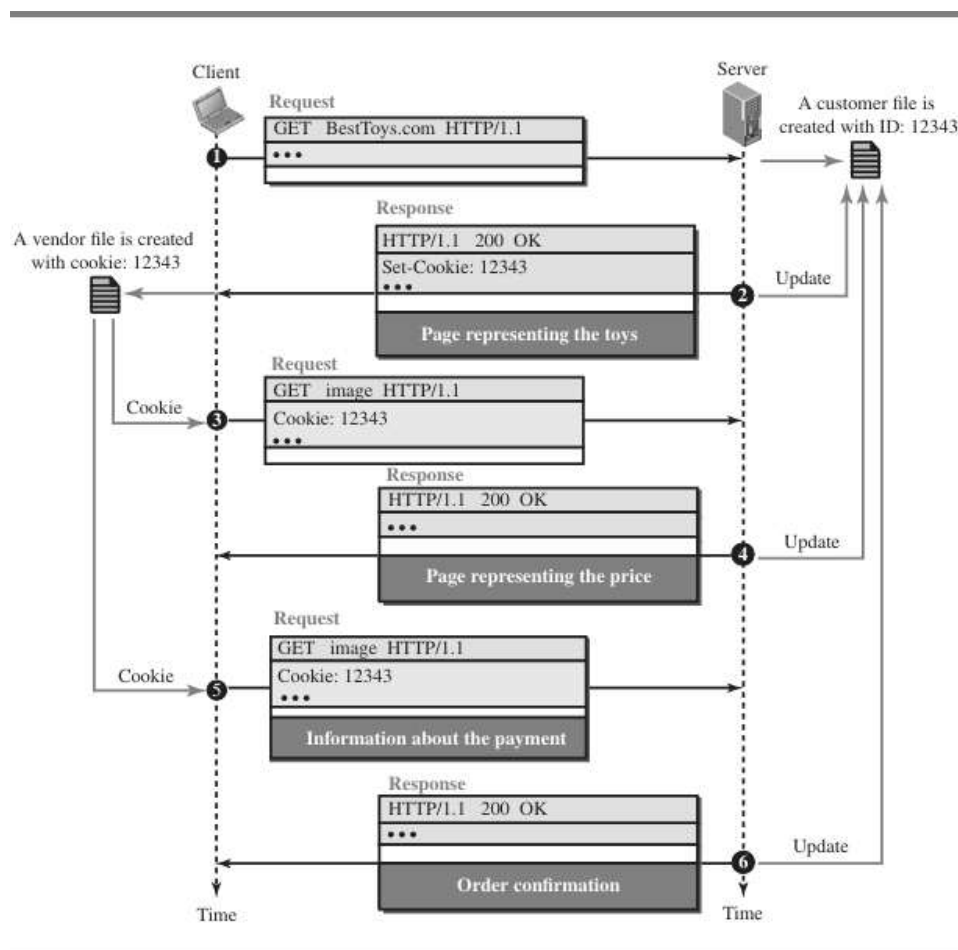
Cookies are widely used in online shopping platforms to maintain **shopping cart contents**, track selected items, and preserve checkout sessions even when users navigate across multiple pages.

- **Web Portals and User Personalization**

Websites such as news portals and dashboards use cookies to store **user preferences**, including language choice, theme selection, and login status, providing a personalized browsing experience.

- **Advertising and Analytics**

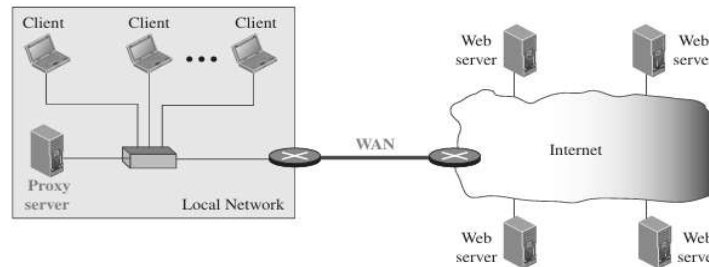
Cookies enable advertisers and analytics platforms to **track user behaviour**, such as visited pages and interaction patterns, across websites. This data is often used for targeted advertising and traffic analysis (subject to privacy regulations).



Web Caching: Proxy Servers

A **Proxy Server** is an intermediary computer that acts as both a server and a client to speed up requests and reduce network traffic.

Figure 26.9 Example of a proxy server



How it works:

1. The client's browser is configured to send all requests to the proxy server, not the destination server.
2. The proxy checks its local **cache**.
 - **Hit:** If the page is in the cache, the proxy returns it immediately (fast!).
 - **Miss:** If not, the proxy acts as a client, fetches the page from the original server, stores a copy in its cache, and then sends it to the user.
3. **Location:** Usually placed at the gateway of a campus or company LAN (to reduce traffic leaving the organization) or at an ISP.

Cache Update: To prevent serving outdated ("stale") pages, proxies use strategies like checking the "Last-Modified" header or keeping a list of frequently updated sites (like news) to refresh often.

5.4 FTP (File Transfer Protocol)

The **File Transfer Protocol (FTP)** is the standard mechanism provided by the TCP/IP protocol suite for copying a file from one host to another. While transferring a file might seem trivial, moving data between two potentially very different systems involves overcoming several incompatibilities:

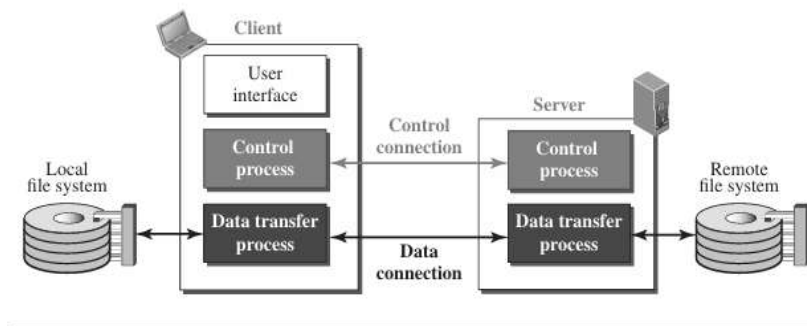
- **File Name Conventions:** Different systems use different rules for naming files.
- **Data Representation:** Systems may use different character sets or data structures.

- **Directory Structures:** The organization of files into folders/directories varies.

FTP solves these problems elegantly, providing a robust method for transferring large files or files with different formats. While HTTP can also transfer files, FTP is specialized for this task and is often the better choice for bulk data transfer.

Architecture

Figure 26.10 FTP



As shown in Figure 26.10, the basic model of FTP is a client-server architecture, but with a unique twist: it separates **control** from **data**.

Components:

1. **Client:**

- **User Interface:** Allows the human user to interact with the system.
- **Client Control Process:** Handles the sending of commands and receiving of responses.
- **Client Data Transfer Process:** Handles the actual movement of file data.

2. **Server:**

- **Server Control Process:** Listens for and executes commands.
- **Server Data Transfer Process:** Manages the data connection for file transfer.

Two Connections

The defining feature of FTP is that it uses **two parallel TCP connections** to transfer a file. This separation makes the protocol more efficient and simpler to implement.

1. The Control Connection:

- **Purpose:** Used for sending commands (like "login", "get file", "change directory") and receiving responses (status codes).
- **Lifetime:** Remains open for the entire duration of the interactive FTP session.
- **Port:** Uses the well-known port **21** on the server.
- **Protocol:** Very simple rules (line-by-line command/response).

2. The Data Connection:

- **Purpose:** Used exclusively for transferring the actual file data.
- **Lifetime:** Transient. It is opened when a file transfer command is issued and **closed** automatically when that specific file transfer is complete. If the user transfers multiple files, the data connection opens and closes multiple times, while the control connection stays open.
- **Port:** Uses the well-known port **20** on the server.
- **Protocol:** More complex rules due to the variety of data types (ASCII, binary, etc.) being transferred.

Control Connection: Communication Details

The control connection uses the NVT ASCII character set (similar to TELNET).

Communication is strictly command-response.

Commands (Client to Server):

Sent as 3 or 4 uppercase ASCII characters, sometimes followed by arguments. (See Table 26.4 for a list).

- **Access Control:** USER (username), PASS (password), QUIT (logout).
- **File Management:** RETR (retrieve/download), STOR (store/upload), DELE (delete), RNFR/RNTO (rename from/to).
- **Directory Management:** CWD (change working directory), LIST (list files), MKD (make directory), RMD (remove directory).
- **Port Management:** PORT (client chooses port), PASV (server chooses port).

Responses (Server to Client):

Every command generates a response consisting of a 3-digit code followed by text. (See

Table 26.5).

- **2xx**: Success (e.g., 200 Command OK, 220 Service ready, 230 User login OK).
- **3xx**: Command accepted, but more information needed (e.g., 331 Username OK; password needed).
- **4xx**: Transient error; try again (e.g., 425 Cannot open data connection).
- **5xx**: Permanent error (e.g., 530 User not logged in, 500 Syntax error).

Table 26.4 Some FTP commands

Command	Argument(s)	Description
ABOR		Abort the previous command
CDUP		Change to parent directory
CWD	Directory name	Change to another directory
DELE	File name	Delete a file
LIST	Directory name	List subdirectories or files
MKD	Directory name	Create a new directory
PASS	User password	Password
PASV		Server chooses a port
PORT	Port identifier	Client chooses a port
PWD		Display name of current directory
QUIT		Log out of the system
RETR	File name(s)	Retrieve files; files are transferred from server to client
RMD	Directory name	Delete a directory
RNFR	File name (old)	Identify a file to be renamed
RNTO	File name (new)	Rename the file
STOR	File name(s)	Store files; file(s) are transferred from client to server
STRU	F, R, or P	Define data organization (F : file, R : record, or P : page)
TYPE	A, E, I	Default file type (A : ASCII, E : EBCDIC, I : image)
USER	User ID	User information
MODE	S, B, or C	Define transmission mode (S : stream, B : block, or C : compressed)

Table 26.5 Some responses in FTP

Code	Description	Code	Description
125	Data connection open	250	Request file action OK
150	File status OK	331	User name OK; password is needed
200	Command OK	425	Cannot open data connection
220	Service ready	450	File action not taken; file not available
221	Service closing	452	Action aborted; insufficient storage
225	Data connection open	500	Syntax error; unrecognized command
226	Closing data connection	501	Syntax error in parameters or arguments
230	User login OK	530	User not logged in

Data Connection: Transferring the File

Creating the data connection is complex because it can be initiated in two ways: **Active** or **Passive**.

1. Active Open (Standard):

- The client opens an **ephemeral port** for data.
- The client sends the PORT command over the control connection to tell the server this port number.
- The **Server initiates** the data connection from its port 20 to the client's specified port.

2. Passive Open (PASV):

- The client sends the PASV command.
- The server opens an ephemeral port and tells the client the number.
- The **Client initiates** the data connection to the server's ephemeral port. (This is often used to bypass client-side firewalls).

Attributes of Transfer:

Before sending data, the client and server must agree on three attributes:

1. File Type:

- **ASCII:** Default. Used for text files.
- **EBCDIC:** Used for IBM mainframes.
- **Image (Binary):** Used for compiled programs, images, etc. The file is sent as a continuous stream of bits without modification.

2. Data Structure:

- **File Structure:** Default. A continuous stream of bytes (no internal structure).
- **Record Structure:** File is divided into records (for text).
- **Page Structure:** File is divided into indexed pages.

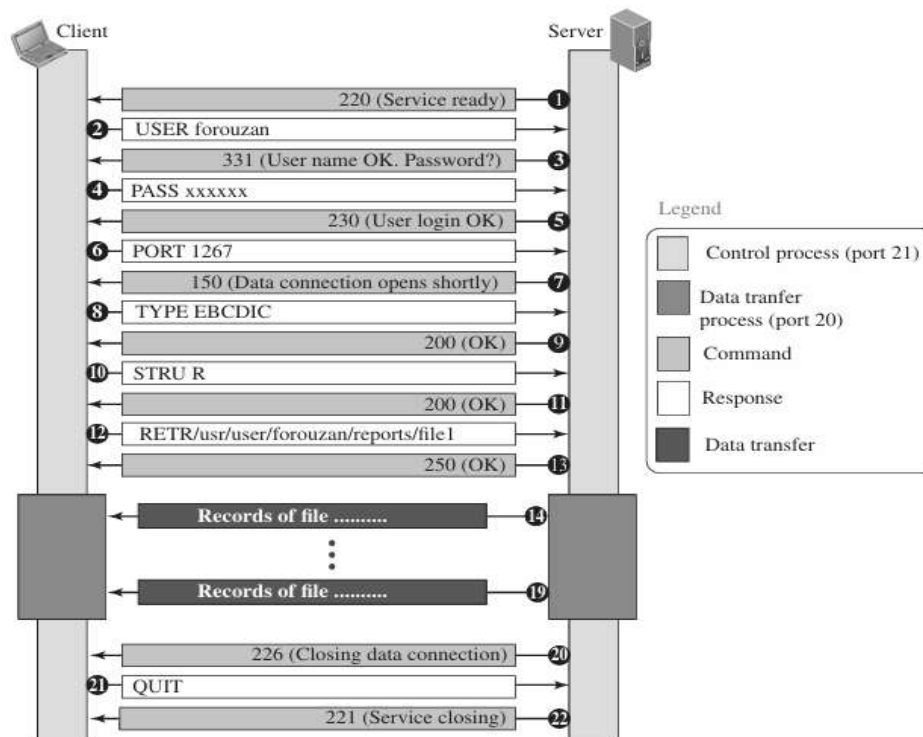
3. Transmission Mode:

- **Stream Mode:** Default. Bytes flow continuously. TCP handles the delivery.
- **Block Mode:** Data is delivered in blocks, each with a 3-byte header.
- **Compressed Mode:** Data is compressed (e.g., run-length encoding) to save bandwidth.

Example FTP Session (Example 26.10 & 26.11)

A typical session involves the following steps:

Figure 26.11 Example 26.10



Example 26.11

The following shows an actual FTP session that lists the directories. The colored lines show the responses from the server control connection; the black lines show the commands sent by the client. The lines in white with black background show data transfer.

```
$ ftp voyager.deanza.fhda.edu
Connected to voyager.deanza.fhda.edu.
220 (vsFTPD 1.2.1)
530 Please login with USER and PASS.
Name (voyager.deanza.fhda.edu:forouzan): forouzan
331 Please specify the password.
Password:*****
230 Login successful.
Remote system type is UNIX.
Using binary mode to transfer files.
227 Entering Passive Mode (153,18,17,11,238,169)
150 Here comes the directory listing.
drwxr-xr-x  2  3027   411   4096  Sep 24  2002  business
drwxr-xr-x  2  3027   411   4096  Sep 24  2002  personal
drwxr-xr-x  2  3027   411   4096  Sep 24  2002  school
226 Directory send OK.
ftp> quit
221 Goodbye.
```

1. **Control Connection Setup:** Client connects to Server Port 21. Server responds 220 Service ready.
2. **Login:** Client sends USER. Server responds 331 Password required. Client sends PASS. Server responds 230 Login successful.
3. **Set Data Specs:** Client might send TYPE I to switch to binary mode ("Image" type).
4. **Data Connection Setup:**
 - Client opens a data port (e.g., ephemeral).
 - Client sends PORT command with its IP and new port.
 - Server responds 200 Command OK.
5. **File Request:** Client sends RETR filename (Retrieve).
6. **Data Transfer:**
 - Server responds 150 Opening data connection.
 - Server opens connection to client's data port (Port 20 -> Ephemeral).
 - File data flows over this new connection.
 - Server closes data connection when done.
 - Server sends 226 Transfer complete over the control connection.
7. **Logout:** Client sends QUIT. Server responds 221 Goodbye and closes the control connection.

Security for FTP

The original FTP protocol was designed in an era of trust and has significant security flaws:

- **Plaintext Passwords:** The USER and PASS commands are sent as clear text. Anyone packet-sniffing the network can steal the login credentials.
- **Plaintext Data:** The file data itself is sent unencrypted.

Solution: To secure FTP, the **Secure Socket Layer (SSL)** (or TLS) is added between the Application layer (FTP) and the Transport layer (TCP). This variation is called **SSL-FTP (FTPS)**. It encrypts both the control commands and the data. Alternatively, **SFTP** (SSH File Transfer Protocol) is a completely different protocol that runs over SSH to provide secure file transfer.

5.5 Electronic Mail (E-mail)

Electronic mail is one of the most widely used applications on the Internet. Its architecture differs significantly from other client-server applications like HTTP or FTP.

Difference from HTTP/FTP:

- **HTTP/FTP:** The server runs continuously. The client initiates a request, and the server responds immediately. It's a synchronous, two-way interaction.
- **E-mail:** It is considered a **one-way transaction**. Alice sends a message to Bob. Bob may not be online. He may reply days later. It is neither feasible nor logical for Bob to run a server program 24/7 just to wait for an email.

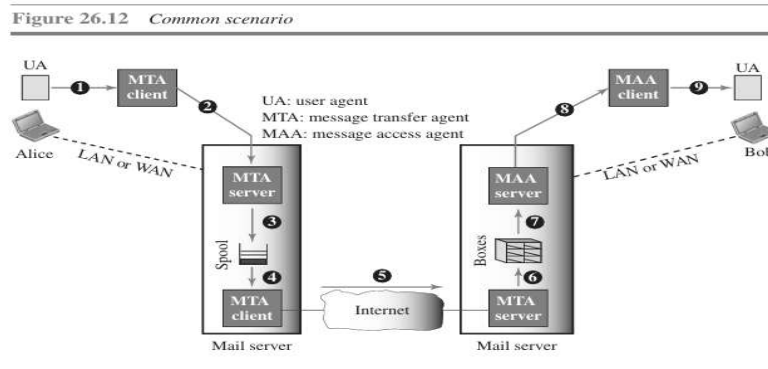
Solution: The system uses **intermediate computers (mail servers)**. Alice sends her mail to a server. The server sends it to Bob's server. Bob retrieves it when he is ready.

Architecture

A simple email delivery involves three distinct types of agents (programs):

1. **User Agent (UA):** The program the user interacts with (e.g., Outlook, Thunderbird, Webmail interface) to compose and read emails.
2. **Message Transfer Agent (MTA):** The server program responsible for moving the email across the Internet from the sender's server to the receiver's server.
3. **Message Access Agent (MAA):** The protocol/program used by the recipient to pull the message from their mailbox on the server to their local device.

The Scenario (Figure 26.12):



1. **Alice (Sender):** Uses a UA to compose a message.
2. **MTA Client (Alice's Server):** The UA sends the mail to a queue (spool) on the local mail server. The MTA client on this server picks it up.
3. **MTA Server (Bob's Server):** The MTA client sends the message over the Internet to the MTA server at Bob's site.
4. **Bob (Receiver):** The MTA server places the email in Bob's mailbox. When Bob is ready, his MAA client (UA) contacts the MAA server to retrieve the mail.

Key Distinction:

- **Push Protocol: SMTP** (used by MTAs). The client pushes the message to the server.
- **Pull Protocol: POP3/IMAP** (used by MAAs). The client pulls the message from the server.

User Agent (UA)

The first component is the User Agent. It provides the interface for composing, reading, and replying to messages.

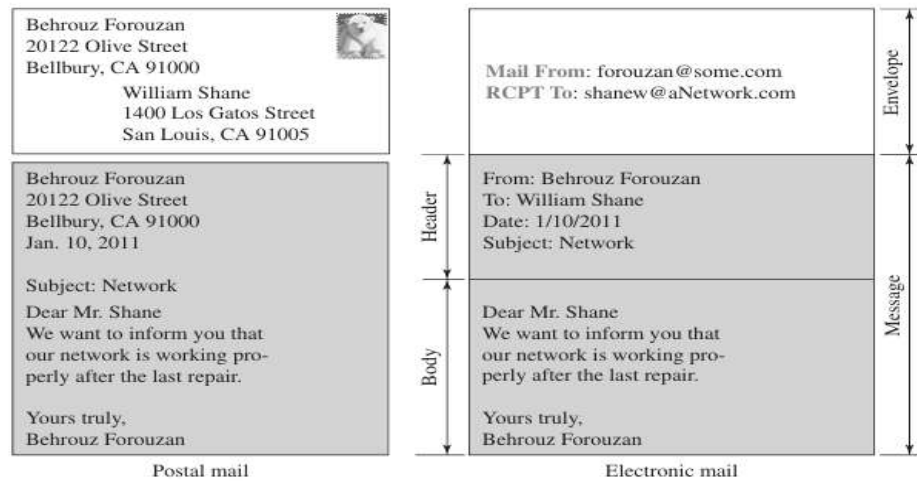
- **Command-Driven:** Early email programs (like mail, pine, elm in UNIX). Users typed single-character commands.
- **GUI-Based:** Modern programs (like *Outlook*, *Eudora*) with windows, icons, and menus.

Sending Mail:

The UA creates a mail packet that mimics postal mail. It consists of:

1. **Envelope:** Contains the sender address, receiver address, and other transmission information. Used by the MTA for routing.
2. **Message:** Contains the actual content. It has two parts:
 - **Header:** Defines the sender, receiver, subject, date, etc.
 - **Body:** The actual text of the message.

Figure 26.13 Format of an e-mail



Receiving Mail

The User Agent (UA) retrieves incoming e-mails either when the user manually checks mail or automatically at scheduled intervals.

- If new mail is available, the UA notifies the user.
- The UA displays a message list, where each entry shows a brief summary such as:
 - Sender's e-mail address
 - Subject of the message
 - Date and time sent or received
- The user can select any message from the list to view its full contents on the screen.

This process allows users to efficiently identify and read received e-mails.

Addressing (Figure 26.14):

Email addresses consist of two parts separated by an @ symbol:

Figure 26.14 E-mail address



- **Local Part:** Defines the user's mailbox file (e.g., forouzan).
- **Domain Name:** Defines the mail server host (e.g., fhda.edu).
- **Result:** forouzan@fhda.edu.

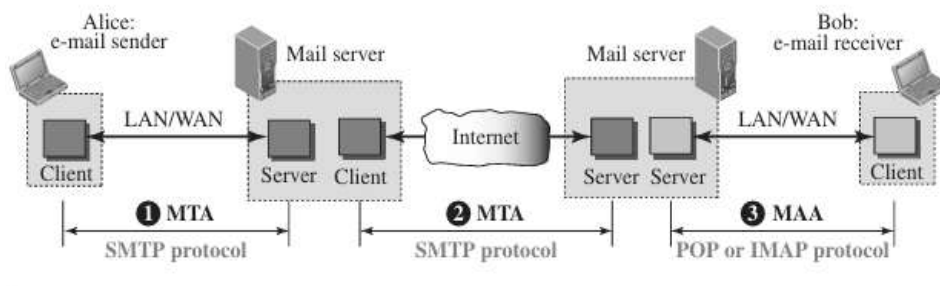
MTA: Simple Mail Transfer Protocol (SMTP)

SMTP is the standard protocol for moving mail from the sender's server to the receiver's server. It operates on **TCP port 25**.

Roles of SMTP:

1. **Between Sender and Sender's Server:** The sender's UA uses SMTP to push the mail to its local server.
2. **Between Servers:** The sender's server (MTA Client) uses SMTP to push the mail to the recipient's server (MTA Server).

Figure 26.15 *Protocols used in electronic mail*



Commands and Responses:

SMTP uses text-based commands (sent by client) and 3-digit numerical responses (sent by server).

- **Commands (Table 26.6):** HELO (identify self), MAIL FROM (sender), RCPT TO (recipient), DATA (start of message body), QUIT (end session).
- **Responses (Table 26.7):** 220 (Service ready), 250 (OK), 354 (Start mail input), 550 (User not local/mailbox unavailable).

Transfer Phases:

1. **Connection Establishment:** Client connects to Server Port 25. Server sends 220.
Client sends HELO. Server sends 250.
2. **Message Transfer:**
 - Client sends MAIL FROM.
 - Client sends RCPT TO.
 - Client sends DATA.
 - Client sends the message lines.
 - Client sends a line with a single period. to signal the end.
3. **Connection Termination:** Client sends QUIT. Server sends 221.

Table 26.6 SMTP commands

Keyword	Argument(s)	Description
HELO	Sender's host name	Identifies itself
MAIL FROM	Sender of the message	Identifies the sender of the message
RCPT TO	Intended recipient	Identifies the recipient of the message
DATA	Body of the mail	Sends the actual message
QUIT		Terminates the message
RSET		Aborts the current mail transaction
VERFY	Name of recipient	Verifies the address of the recipient
NOOP		Checks the status of the recipient
TURN		Switches the sender and the recipient
EXPN	Mailing list	Asks the recipient to expand the mailing list
HELP	Command name	Asks the recipient to send information about the command sent as the argument
SEND FROM	Intended recipient	Specifies that the mail be delivered only to the terminal of the recipient, and not to the mailbox
SMOL FROM	Intended recipient	Specifies that the mail be delivered to the terminal <i>or</i> the mailbox of the recipient
SMAL FROM	Intended recipient	Specifies that the mail be delivered to the terminal <i>and</i> the mailbox of the recipient

Table 26.7 Responses

Code	Description
Positive Completion Reply	
211	System status or help reply
214	Help message
220	Service ready
221	Service closing transmission channel
250	Request command completed
251	User not local; the message will be forwarded
Positive Intermediate Reply	
354	Start mail input
Transient Negative Completion Reply	
421	Service not available
450	Mailbox not available
451	Command aborted; local error
452	Command aborted; insufficient storage
Permanent Negative Completion Reply	
500	Syntax error; unrecognized command

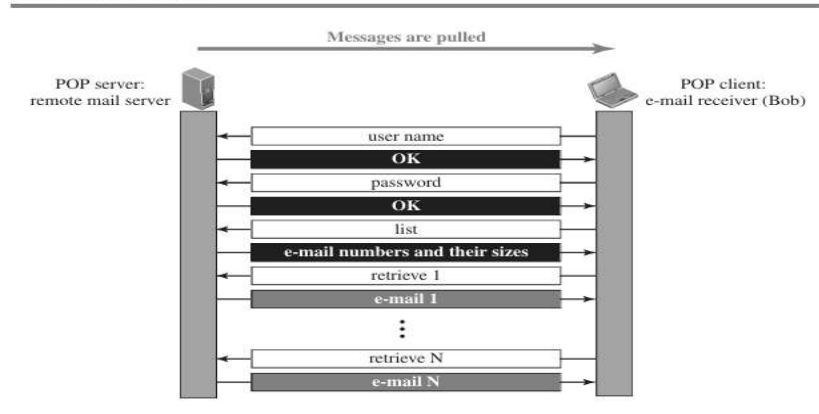
Code	Description
501	Syntax error in parameters or arguments
502	Command not implemented
503	Bad sequence of commands
504	Command temporarily not implemented
550	Command is not executed; mailbox unavailable
551	User not local
552	Requested action aborted; exceeded storage location
553	Requested action not taken; mailbox name not allowed
554	Transaction failed

MAA: POP3 and IMAP4

SMTP is a **push** protocol; it cannot be used by the receiver to retrieve mail because the receiver's computer might be off, or they might want to pull mail at their convenience. For this, we use **Message Access Agents**.

1. POP3 (Post Office Protocol, version 3)

Figure 26.17 POP3



- **Function:** Simple and limited. It is designed to download mail from the server to the client's local machine.
- **Port:** TCP Port 110.
- **Modes:**
 - **Delete Mode:** Mail is deleted from the server after retrieval. (Good for storage, bad if you use multiple computers).
 - **Keep Mode:** Mail remains on the server after retrieval.
- **Workflow:** Connect -> Login (USER/PASS) -> Retrieve (RETR) -> Delete (DELE) -> Quit.

2. IMAP4 (Internet Mail Access Protocol, version 4)

- **Function:** More powerful and complex than POP3. It allows users to manage mail **on the server**.
- **Features:**
 - Users can create, delete, and rename folders/mailboxes *on the server*.
 - Users can check the email header before downloading the body (saves bandwidth).

- Users can search for specific strings in emails on the server.
- Users can partially download emails (e.g., just the text, not the heavy attachment).
- **Use Case:** Ideal for users who access their email from multiple different devices (laptop, phone, work computer) and want a synchronized view.

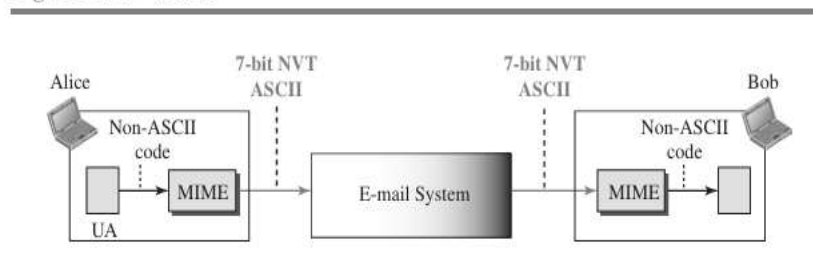
MIME (Multipurpose Internet Mail Extensions)

The original SMTP protocol had a major limitation: it could only handle **7-bit NVT ASCII** text. It could not send:

- Languages other than English (French accents, Chinese characters, etc.).
- Binary files (Images, Video, Audio, Executables).

Solution: MIME is a supplementary protocol that allows non-ASCII data to be sent through SMTP.

Figure 26.18 MIME



Mechanism:

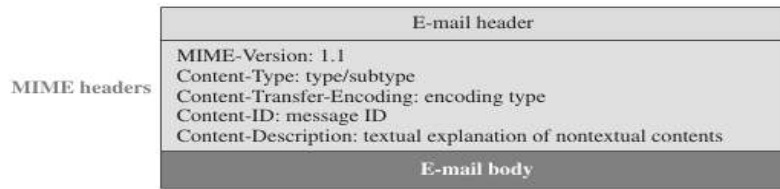
MIME acts as a translator.

1. **Sender Side:** MIME transforms non-ASCII data (e.g., an image) into 7-bit NVT ASCII data.
2. **Transmission:** The data is sent over the standard SMTP protocol.
3. **Receiver Side:** MIME transforms the ASCII data back into the original non-ASCII data.

MIME Headers:

MIME adds 5 headers to the email to define the transformation:

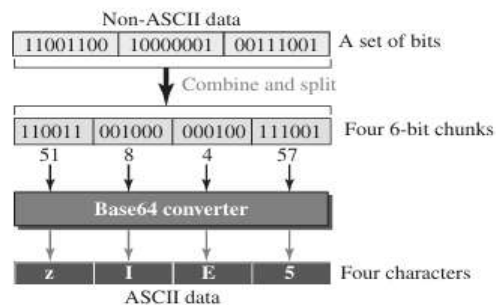
Figure 26.19 MIME header



1. **MIME-Version:** (e.g., 1.1).
2. **Content-Type:** Defines the data type (e.g., text/html, image/jpeg, video/mpeg).
3. **Content-Transfer-Encoding:** How the data was converted to ASCII.
 - **7-bit:** No conversion needed.
 - **Base64:** Used for binary data. Maps every 6 bits of input to one printable ASCII character.
 - **Quoted-Printable:** Used for mostly text data with a few non-ASCII characters.
4. **Content-Id:** Uniquely identifies the message.
5. **Content-Description:** Describes the body (e.g., "My Picture").

Base64 Encoding (Figure 26.20):

Figure 26.20 Base64 conversion



- Converts binary data into a safe ASCII subset (A-Z, a-z, 0-9, +, /).
- **Process:**
 1. Take a block of 24 bits (3 bytes) of binary data.
 2. Divide it into four 6-bit chunks.
 3. Map each 6-bit chunk to a character in the Base64 table.
- **Result:** 3 bytes of binary data become 4 bytes of ASCII text (33% overhead).

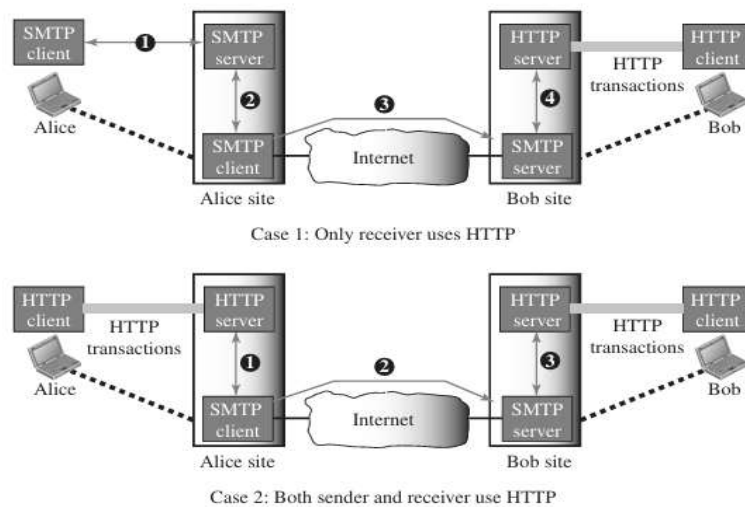
Web-Based Mail

Web-based e-mail services (such as **Gmail, Yahoo Mail, and Outlook/Hotmail**) allow users to send and receive e-mails through a web browser using **HTTP**, instead of traditional mail clients.

Case I: Only Receiver Uses Web-Based Mail

- The sender (Alice) uses a **traditional mail server**.
- Mail transfer from Alice to her mail server uses **SMTP**.
- Transfer between mail servers also uses **SMTP**.
- The receiver (Bob) accesses e-mail via a **web browser**.
- Bob retrieves messages from the web server using **HTTP** (instead of POP3/IMAP).
- E-mails are displayed in **HTML format** after successful login.

Figure 26.22 Web-based e-mail, cases I and II



Case II: Both Sender and Receiver Use Web-Based Mail

- Both Alice and Bob use **web-based e-mail services**.
- Alice sends the e-mail to her web server using **HTTP**.
- The sending web server forwards the message to Bob's web server using **SMTP**.

- Bob reads the message using **HTTP** through his browser.
- **SMTP is still used between mail servers**, even though users interact via HTTP.

E-Mail Security

Basic e-mail protocols such as **SMTP, POP3, and IMAP** do **not provide built-in security**.

To ensure confidentiality, integrity, and authentication, application-layer security protocols are used:

- **Pretty Good Privacy (PGP)**
- **Secure/Multipurpose Internet Mail Extensions (S/MIME)**

These protocols provide **encryption, digital signatures, and authentication** for secure e-mail communication.

5.6 TELNET (Terminal Network)

A fundamental capability of the Internet is allowing a user on one computer to access resources and programs on another computer, potentially thousands of miles away. While it is impossible to write a custom client-server pair for every single application, we can solve this by allowing a user to **log into** the remote computer and use its services as if they were sitting right in front of it.

This concept is known as **Remote Logging**.

The Challenge: Heterogeneity

Connecting two computers for remote logging is complex because operating systems (OS) are different.

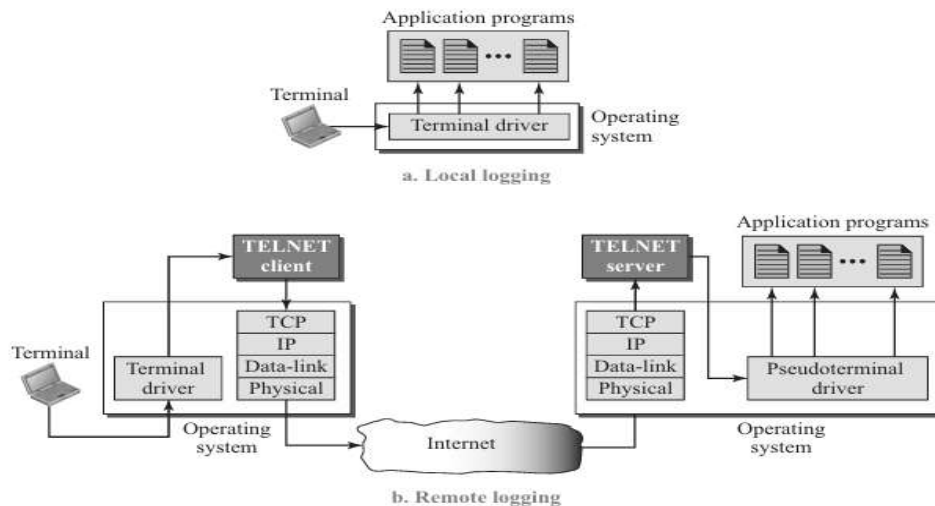
- A computer running **DOS** sends Ctrl+z to signal the "end of file."
- A computer running **UNIX** sends Ctrl+d to signal the same thing.
- Keyboards and terminal displays handle characters differently.

If a user on a Macintosh tries to log into a UNIX server, the server won't understand the Mac's specific keystrokes. To solve this, we need a universal language.

Local versus Remote Logging

To understand TELNET, we must distinguish between logging into a local machine and a remote one.

Figure 26.23 Local versus remote logging



1. Local Logging (Figure 26.23a)

- **Scenario:** A user sits at a terminal directly connected to the computer.
- **Process:**
 1. User types a keystroke.
 2. The **Terminal Driver** accepts the character.
 3. The Terminal Driver passes the character to the **Operating System (OS)**.
 4. The OS interprets the character (e.g., if it's a command) and invokes the appropriate application program.

2. Remote Logging (Figure 26.23b)

- **Scenario:** A user wants to access an application on a remote machine via the Internet.
- **Process:**
 1. User types a keystroke.
 2. The local OS accepts the character but **does not** interpret it.
 3. The character is sent to the **TELNET Client**.

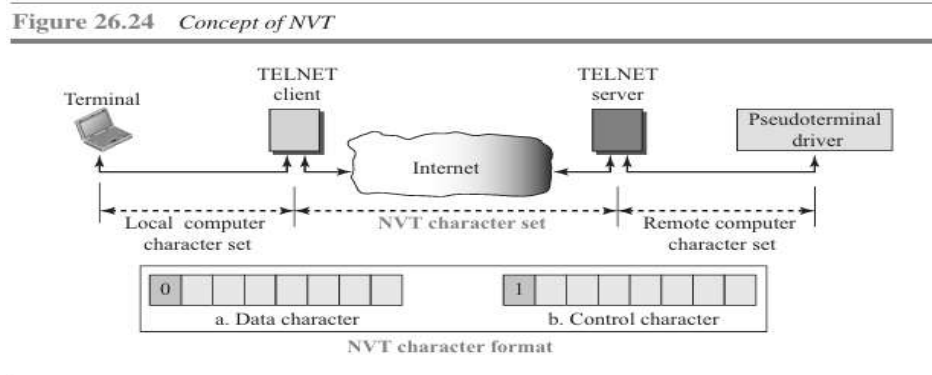
4. The TELNET Client translates the character into a universal format called **Network Virtual Terminal (NVT)**.
5. The NVT character travels across the Internet (via TCP/IP) to the remote machine.
6. The **TELNET Server** receives the NVT character and translates it into a format the remote OS understands.
7. The character is passed to a **Pseudoterminal Driver** (software that pretends to be a physical terminal).
8. The remote OS receives the character from the pseudoterminal driver and processes it as if it came from a local keyboard.

Network Virtual Terminal (NVT)

The NVT is the universal "language" of TELNET. It bridges the gap between heterogeneous systems.

- **Client Side:** Translates local characters -> NVT form.
- **Server Side:** Translates NVT form -> Remote computer's native form.

NVT Character Set (Figure 26.24):



NVT uses an 8-bit byte for data and control.

1. **Data Characters:** Based on **NVT ASCII**. It is an 8-bit set where the 7 lowest bits are standard US ASCII, and the highest bit is 0. (e.g., 'A', 'b', '1').
2. **Control Characters:** Used to send commands (like "Interrupt Process" or "Are You There"). In these bytes, the highest bit is set to 1.

Options and Negotiation

TELNET is not just a dumb terminal; it is sophisticated. It allows the client and server to negotiate **Options** to enhance the session.

- **Example:** A user with a sophisticated terminal might want to use advanced features (like specific cursor movements), while a user with a simple terminal uses default features.
- **Negotiation:** Before or during the session, the client and server exchange commands to agree on which options (e.g., 7-bit vs 8-bit data, echo modes) will be used.

Security

TELNET is insecure.

- It sends **everything**—including the username and password—in **plaintext**.
- A hacker eavesdropping on the network can easily steal login credentials.
- **Modern Solution:** TELNET has largely been replaced by **SSH (Secure Shell)**, which encrypts the entire session, including the login process.
- **Why study TELNET?** It is the foundation for understanding remote logging, and network administrators still use it occasionally for debugging network services (e.g., manually connecting to a web server on port 80).

Summary Table: TELNET Components

Component	Function
TELNET Client	Runs on the user's machine. Translates local keystrokes to NVT.
TELNET Server	Runs on the remote machine. Translates NVT to remote OS commands.
NVT	The universal 8-bit character set used for transmission.
Pseudoterminal Driver	Software on the remote OS that mimics a physical terminal input.

5.7 Secure Shell (SSH)

Secure Shell (SSH) is a secure application-layer protocol primarily used for **remote logging** and **file transfer**. It was originally designed as a secure replacement for insecure protocols like TELNET and older remote shell commands (rsh, rlogin, rexec).

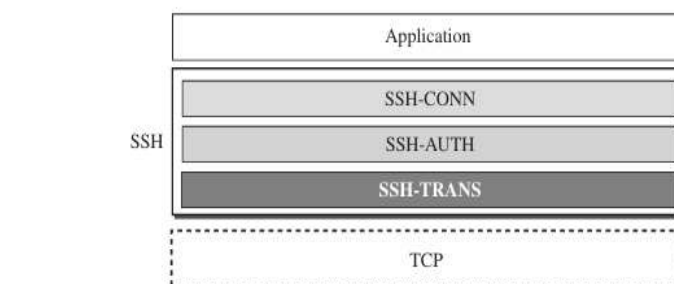
Versions:

- **SSH-1:** The original version. It is now deprecated due to security flaws.
- **SSH-2:** The current standard. It is totally incompatible with SSH-1 and offers robust security features.

Components of SSH

SSH is not a single monolithic protocol; it is an application-layer protocol suite comprised of three distinct components (as shown in Figure 26.25) that layer on top of each other:

Figure 26.25 Components of SSH



1. **SSH-TRANS:** The Transport-Layer Protocol.
2. **SSH-AUTH:** The Authentication Protocol.
3. **SSH-CONN:** The Connection Protocol.

These components work together on top of TCP to provide a secure service.

1. SSH Transport-Layer Protocol (SSH-TRANS)

TCP is reliable but insecure. The first job of SSH is to create a secure, encrypted tunnel on top of TCP. SSH-TRANS is an independent protocol that handles the initial setup of this secure channel.

Process:

1. **TCP Connection:** The client and server establish a standard TCP connection.
2. **Security Negotiation:** They exchange parameters to agree on encryption algorithms.
3. **Secure Channel Establishment:** A secure, encrypted channel is created.

Services Provided by SSH-TRANS:

- **Privacy/Confidentiality:** All messages exchanged are encrypted, preventing eavesdropping.
- **Data Integrity:** Messages are checked to ensure they have not been altered by an intruder during transit.
- **Server Authentication:** The client verifies the server's identity, ensuring it is connecting to the real server and not an imposter (avoiding Man-in-the-Middle attacks).
- **Compression:** Messages can be compressed to improve network efficiency and make certain attacks harder.

2. SSH Authentication Protocol (SSH-AUTH)

Once the secure tunnel is built by SSH-TRANS, the server needs to know who the client is. SSH-AUTH handles the **Client Authentication**.

Process:

1. Starts after the secure channel is established.
2. The client sends a request message containing the username, server name, method of authentication (e.g., password, public key), and required data.
3. The server verifies the credentials.
4. The server responds with a success message or a failure message.

Note: This step is similar to what SSL (Secure Socket Layer) does, providing a distinct layer for verifying the user's identity.

3. SSH Connection Protocol (SSH-CONN)

After authentication is complete, the two parties are ready to do useful work. SSH-CONN runs on top of the secure, authenticated channel provided by the previous two layers.

Primary Function: Multiplexing

SSH-CONN allows the client to create **multiple logical channels** over the single SSH connection. Each channel can be used for a different purpose simultaneously.

Examples of Channels:

- **Remote Logging:** Interactive shell session.
- **File Transfer:** Using SFTP (SSH File Transfer Protocol).
- **Port Forwarding:** Tunneling other applications (like X11 graphics) through the secure SSH connection.

Applications of SSH

SSH is a versatile tool used for several key network administration tasks.

1. Remote Logging:

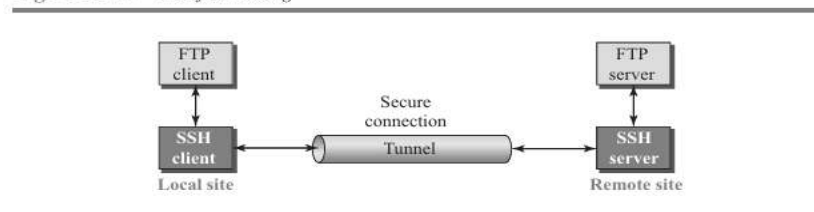
- This is the most common use. It replaces TELNET.
- It provides a secure environment for a user to log into a remote server and execute commands.

2. File Transfer (SFTP):

- SSH includes a subsystem called **SFTP (SSH File Transfer Protocol)**.
- It provides secure file access, transfer, and management.
- Unlike standard FTP, SFTP encrypts both commands and data, preventing passwords and sensitive files from being exposed to packet sniffers.

3. Port Forwarding (Tunnelling):

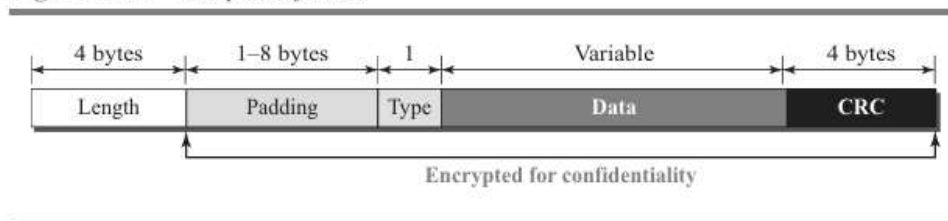
Figure 26.26 Port forwarding



- SSH can secure *other* application protocols that are not secure by themselves.
- **Mechanism:** SSH creates a tunnel. An insecure application (like an old database client) sends its data to the local SSH client. The SSH client encrypts the data, sends it through the tunnel to the SSH server, which decrypts it and forwards it to the destination application.
- This effectively wraps insecure traffic in a layer of strong encryption.

Format of SSH Packets:

Figure 26.27 SSH packet format



An SSH packet has a fixed structure to ensure **secure and error-free communication**.

- **Length (4 bytes):** Specifies the size of the packet (excluding padding).
- **Padding (1-8 bytes):** Added to improve security and make attacks harder.
- **Type (1 byte):** Identifies the kind of SSH packet (data, control, etc.).
- **Data (variable):** Actual information being transmitted.
- **CRC (4 bytes):** Used for error detection.

Summary of SSH Architecture

Component	Role	Layer Position
Applications	Remote login, File transfer, etc.	Top
SSH-CONN	Multiplexes multiple logical channels.	Middle (Upper)
SSH-AUTH	Authenticates the client user.	Middle (Lower)
SSH-TRANS	Encrypts data, ensures integrity, authenticates server.	Bottom (Top of TCP)
TCP	Provides reliable transport.	Transport Layer

Domain Name System (DNS)

To identify a computer on the Internet, the TCP/IP protocol suite uses an **IP address** (32-bit for IPv4, 128-bit for IPv6). This unique numerical identifier is perfect for machines but difficult for humans to remember. People prefer names (like www.google.com) over numbers (like 142.250.183.68).

This creates a need for a directory system that can map a human-readable name to a machine-readable IP address. This is analogous to a telephone book: you look up a person's name to find their phone number. On the Internet, this system is the **Domain Name System (DNS)**.

The Challenge of Scale

In the early days of the Internet (ARPANET), a single central file (hosts.txt) stored all name-to-address mappings. However, as the Internet grew, a centralized approach became impossible because:

1. **Traffic:** A single central server would be overwhelmed by requests.
2. **Reliability:** If the central computer failed, the entire network's ability to resolve names would collapse.
3. **Latency:** A single server cannot be "close" to all users globally.

Solution: Distribute the information among many computers across the world. The host needing a mapping contact the closest computer holding the needed information.

Purpose of DNS

The primary purpose of the Domain Name System (DNS) is to translate human-readable host names into machine-readable IP addresses so that Internet applications can function correctly.

In the TCP/IP protocol suite:

- Machines identify hosts using **IP addresses**
 - **IPv4:** 32-bit address
 - **IPv6:** 128-bit address

- Humans prefer **symbolic names** such as `www.google.com`

DNS acts as a **directory service**, mapping:

Host Name $\leftrightarrow$ IP Address

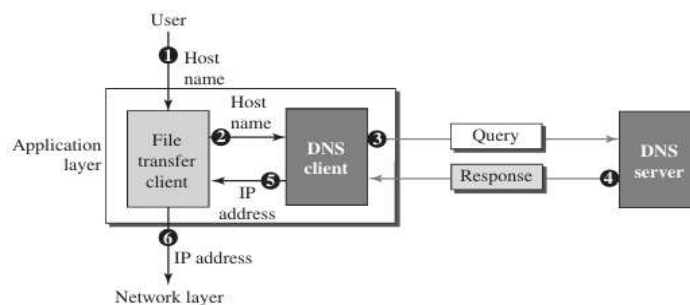
This function is essential because:

- Applications such as **HTTP, FTP, SMTP, SSH** require IP addresses
- Users only know **domain names**, not numeric IPs

Analogy:

DNS is similar to a **telephone directory**, where a person's name is mapped to a phone number.

Figure 26.28 Purpose of DNS



Name Space

To prevent ambiguity, every name assigned to a machine must be unique. A **Name Space** maps each address to a unique name. There are two ways to organize a name space:

- Flat Name Space:** A sequence of characters without structure (e.g., "printer1", "marketing-pc").
 - Problem:** Must be centrally controlled to avoid duplicates. Not scalable for the Internet.
- Hierarchical Name Space:** Used by DNS. Each name is made of several parts (e.g., `challenge.at.fhda.edu`).
 - Structure:** The first part defines the specific host, the second part the department, the third part the organization type, etc.

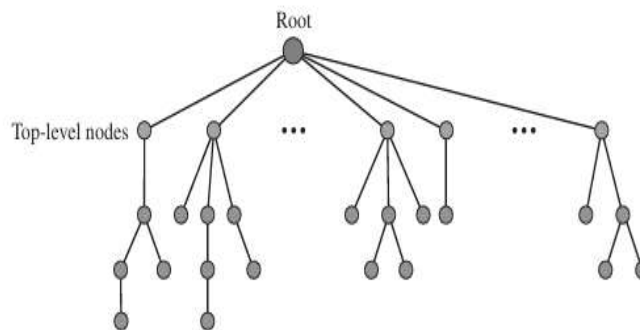
- **Advantage:** Decentralized control. A central authority assigns the high-level part (fhda.edu), and the organization (fhda) is then free to assign sub-names (at.fhda.edu, cs.fhda.edu) without checking with the central authority.

Domain Name Space

DNS uses an **inverted-tree structure** to manage its hierarchical name space.

- **Root:** The top of the tree is the **root** (level 0).
- **Levels:** The tree can have up to 128 levels (0 to 127).
- **Label:** Each node in the tree has a label (a string up to 63 characters). The root label is a null string (empty).
- **Domain Name:** The full name of a node is the sequence of labels read from the node **up to the root**, separated by dots.
 - **Example:** uci.edu. (Note: technically ends with a dot for the root).

Figure 26.29 Domain name space



FQDN vs. PQDN:

- **Fully Qualified Domain Name (FQDN):** A complete domain name ending in a null label (dot). It uniquely identifies a host. Example: challenger.at.fhda.edu.
- **Partially Qualified Domain Name (PQDN):** Starts from a node but does not reach the root. Used for convenience within a local site (e.g., just typing challenger when you are already in the fhda.edu network).

Figure 26.30 Domain names and labels

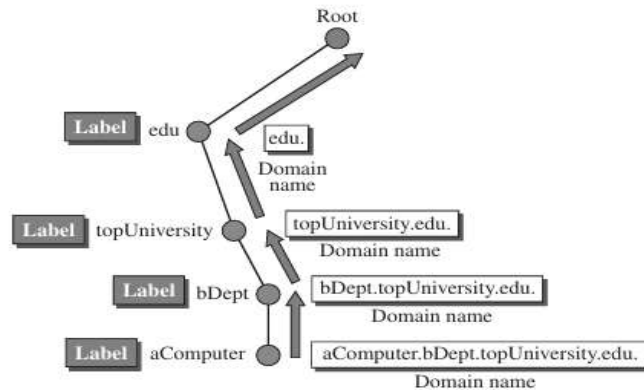
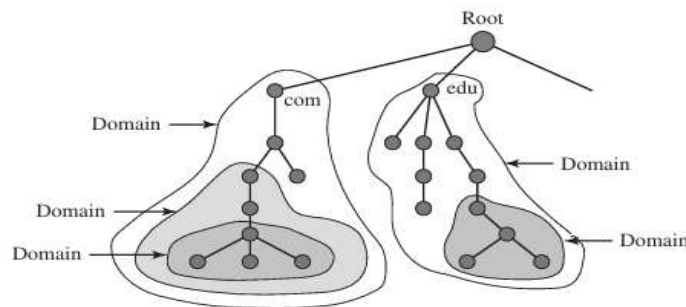


Figure 26.31 Domains



26.6.3 Distribution of Name Space

The DNS tree is not stored on one server. It is divided into **zones**.

- **Domain:** A subtree of the domain name space. (fig 26.31)
- **Zone:** The specific part of a domain that a single server is responsible for (has authority over). (fig 26.33)
- **Root Server:** A server whose zone is the entire tree. It delegates authority to top-level domains (like .com, .edu). There are 13 logical root server clusters worldwide.
- **Primary Server:** Stores a file with the authoritative information for a zone. Responsible for creating, maintaining, and updating the zone file on its local disk.
- **Secondary Server:** Stores a copy of the zone info transferred from a primary (or other secondary) server. It provides redundancy.

Figure 26.32 Hierarchy of name servers

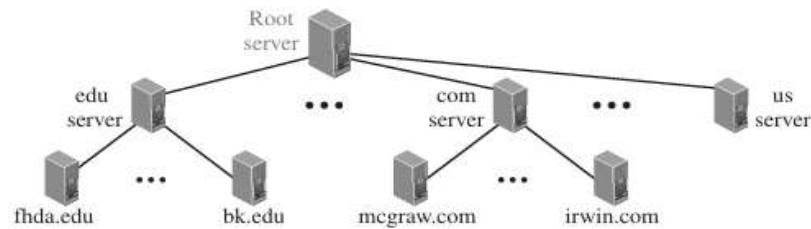
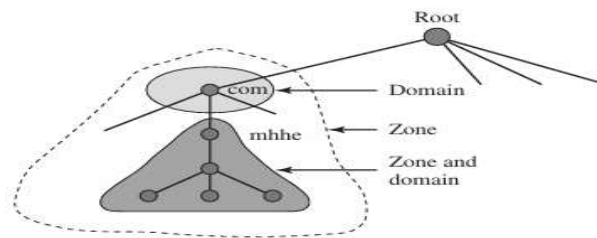


Figure 26.33 Zone



DNS in the Internet

The Internet domain name space is divided into three main sections:

1. **Generic Domains:** Define registered hosts according to their generic behaviour.
 - o com (Commercial), edu (Education), gov (Government), org (Non-profit), net (Network support), mil (Military), int (International).

Figure 26.34 Generic domains

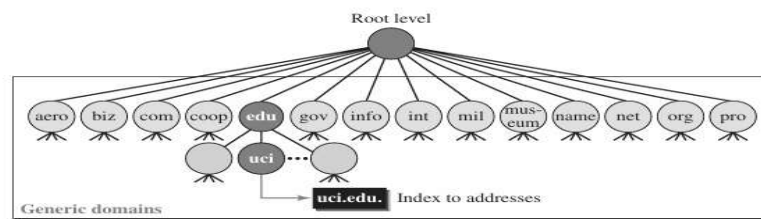
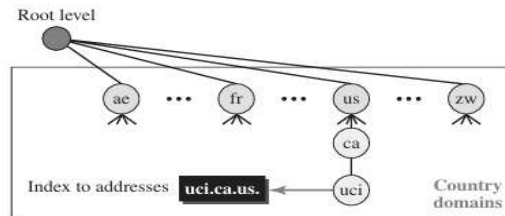


Table 26.12 Generic domain labels

Label	Description	Label	Description
aero	Airlines and aerospace	int	International organizations
biz	Businesses or firms	mil	Military groups
com	Commercial organizations	museum	Museums
coop	Cooperative organizations	name	Personal names (individuals)
edu	Educational institutions	net	Network support centers
gov	Government institutions	org	Nonprofit organizations
info	Information service providers	pro	Professional organizations

2. **Country Domains:** Use two-character country codes (e.g., us, uk, fr, in).
 - Example: uci.ca.us (University of California, Irvine, California, United States).

Figure 26.35 Country domains



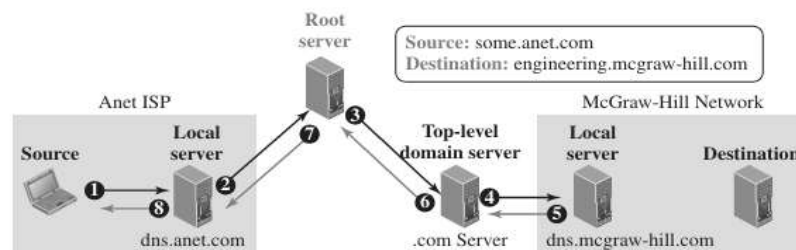
3. **Inverse Domains:** Used to map an address to a name (IP -> Name).

Resolution

Mapping a name to an address is called **Name-Address Resolution**.

- **Resolver:** The client program on a host that needs a mapping. It contacts the closest DNS server.

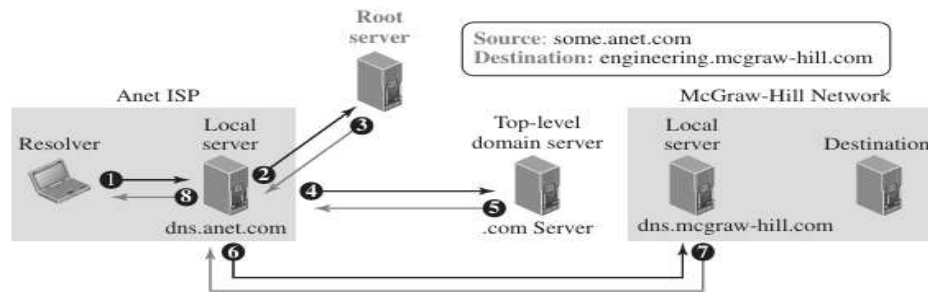
Figure 26.36 Recursive resolution



Recursive Resolution (Figure 26.36):

1. Host asks Local DNS Server.
 2. If Local Server doesn't know, it asks a Root Server.
 3. Root Server asks Top-Level Domain (TLD) Server (e.g., .com server).
 4. TLD Server asks Authoritative Server (e.g., mcgraw-hill.com).
 5. The answer bubbles all the way back to the Host.
- **Key:** The burden of finding the answer is placed on the servers.

Figure 26.37 Iterative resolution

**Iterative Resolution (Figure 26.37):**

1. Host asks Local DNS Server.
2. Local Server asks Root Server. Root says, "I don't know, but here is the IP of the .com server."
3. Local Server asks .com server. It says, "I don't know, but here is the IP of mcgraw-hill.com server."
4. Local Server asks mcgraw-hill.com server. It gets the answer.
5. Local Server gives answer to Host.

- **Key:** The Local Server does the running around.

Caching:

To reduce traffic, servers **cache** (store) the answers they receive. If another user asks for google.com, the server provides the cached answer without querying the global network again. Caches have a **TTL (Time to Live)** to prevent stale data.

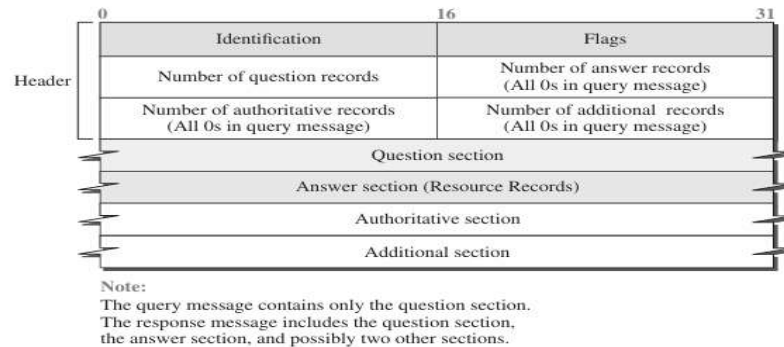
DNS Messages

DNS uses a single message format for both Queries (Requests) and Responses.

- **Header:** Contains Identification (to match request/response) and Flags (Query vs Response, Recursive desired, etc.).
- **Question Section:** Contains the name being queried and the type of query.
- **Answer Section:** Contains **Resource Records** (the IP address mapping).
- **Authoritative/Additional Sections:** Info about the server providing the answer.

Transport: DNS uses **UDP port 53** for most standard queries (fast, low overhead) and **TCP port 53** if the response data exceeds 512 bytes (e.g., zone transfers).

Figure 26.38 DNS message



Registrars

- **ICANN:** The global authority that manages the top-level domains.
- **Registrar:** A commercial entity accredited by ICANN (e.g., GoDaddy, Namecheap). They verify the uniqueness of a requested domain name and enter it into the DNS database for a fee.

DDNS (Dynamic DNS)

Standard DNS involves manually updating zone files. This is too slow for today's dynamic environment (e.g., DHCP assigning new IPs daily).

- **DDNS:** Allows a client (like a DHCP server) to automatically and dynamically update the primary DNS server when an IP address changes.

DNS Security

DNS is critical infrastructure but was designed without security.

- **Attacks:**
 1. Reading responses to map a network's internal structure.
 2. **DNS Spoofing/Cache Poisoning:** An attacker sends a fake response to a DNS server, tricking it into caching a wrong IP (e.g., directing bank.com to a hacker's site).

3. Denial of Service (DoS) attacks on root servers.

- **Solution (DNSSEC):** Adds digital signatures to DNS records to verify **Data Origin Authentication** (the answer came from the real server) and **Data Integrity** (the answer wasn't changed). It does *not* provide confidentiality (encryption).